CENTRO UNIVERSITÁRIO UNIFIO · ENGENHARIA DE SOFTWARE · PROJETO INTEGRADOR

Conecta Event — Documentação Técnica e Diagrama de Classes UML

Integrantes: Ana Carolina Medeiro, Ana Clara Correa, Kauã Yury e Lucas Scalada Noesse | **Repositório Git:** github.com/Niesiloko/Projeto-Integrador-Final

1. Visão Geral da Arquitetura do Sistema

O sistema **Conecta Event** (README.md) é uma plataforma web para publicação de eventos, venda de ingressos identificados por código único (QR Code) e gestão do ciclo de reembolsos com painel de indicadores operacionais para organizadores.

Camada de Apresentação (Frontend Angular)

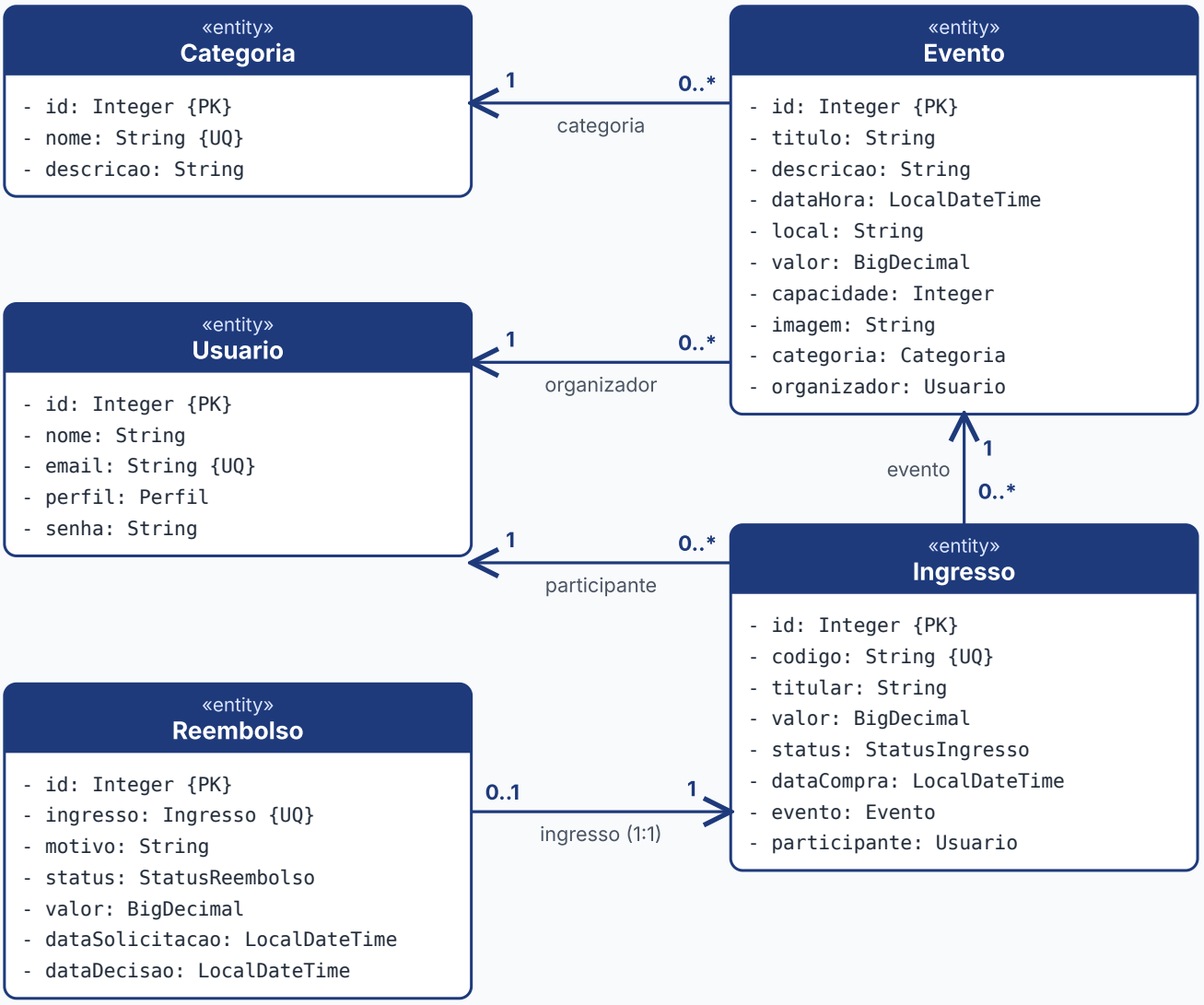
Construída em Angular 22 (package.json) com `standalone: false`, arquitetura modular (`AppModule` e `EventosModule`), roteamento SPA sem SSR/SSG (angular.json) e componentes do Angular Material.

Camada de Domínio e Persistência (Backend Spring Boot)

Desenvolvida em Java 21 com Spring Boot, Spring Data JPA, Jakarta Bean Validation e MySQL (backend/pom.xml), estruturando cinco entidades principais e três tipos enumerados em br.edu.unifio.conectaevent.entidades.

2. Diagrama de Classes UML do Domínio

A Figura 1 apresenta o Diagrama de Classes UML com as cinco classes de entidade persistentes (`Usuario`, `Categoria`, `Evento`, `Ingresso` e `Reembolso`), as três enumerações (`Perfil`, `StatusIngresso` e `StatusReembolso`) e suas associações com multiplicidades e navegabilidade UML.



«enum» Perfil
ORGANIZADOR
PARTICIPANTE

«enum» StatusReembolso
PENDENTE | APROVADO
RECUSADO

«enum» StatusIngresso
CONFIRMADO | UTILIZADO
REEMBOLSO_SOLICITADO
REEMBOLSADO

Leitura das associações: a seta indica a **navegabilidade** — ela parte da classe que guarda a referência e aponta para a classe referenciada (ex.: `Evento` possui o atributo `organizador: Usuario`, por isso a seta vai de Evento para Usuario). O número junto a cada classe é a multiplicidade daquele lado: um Usuario organiza **0..*** eventos, e cada Evento tem exatamente **1** organizador.

Figura 1 — Diagrama de Classes UML das entidades e enumerações do Conecta Event (pacote de entidades).

3. Especificação Técnica das Entidades e Enumerações

Todas as classes de domínio utilizam anotações JPA (`@Entity`, `@Id`, `@GeneratedValue`), Lombok (`@Getter`, `@Setter`) e validações do Jakarta Bean Validation:

Entidade / Arquivo	Atributo e Tipo Java	Mapeamento JPA e Regras de Validação

Usuario
Usuario.java
	`id : Integer`
	`nome : String`
	`email : String`
	`perfil : Perfil`
	`senha : String`
		PK (`@Id`, `IDENTITY`); `nome` obrigatório (3 a 100 chars); `email` válido e único (máx. 120 chars, `unique = true`); `perfil` obrigatório (`@Enumerated(STRING)`); `senha` de 6 a 100 chars, gravada como hash BCrypt e oculta na leitura JSON (`WRITE_ONLY`).

Categoria
Categoria.java
	`id : Integer`
	`nome : String`
	`descricao : String`
		PK (`@Id`, `IDENTITY`); `nome` obrigatório (2 a 50 chars) e único (`unique = true`); `descricao` opcional limitada a 255 caracteres (`@Size(max = 255)`).

Evento
Evento.java
	`id : Integer`
	`titulo / descricao`
	`dataHora : LocalDateTime`
	`local : String`
	`valor / capacidade / imagem`
	`categoria / organizador`
		PK (`IDENTITY`); `titulo` (3 a 120 chars); `descricao` (até 2000 chars); `dataHora` obrigatória no futuro (`@Future`); `local` na coluna `local_evento` (máx. 150); `valor` (≥ 0.00); `capacidade` (1 a 100000); FKs `@ManyToOne` obrigatórias para `Categoria` e `Usuario`.

Ingresso
Ingresso.java
	`id : Integer`
	`codigo : String`
	`titular : String`
	`valor : BigDecimal`
	`status : StatusIngresso`
	`dataCompra : LocalDateTime`
	`evento / participante`
		PK (`IDENTITY`); `codigo` único para QR Code (`unique = true`); `titular` obrigatório (3 a 100 chars); `valor` e `dataCompra` gravados na emissão; `status` via `@Enumerated(STRING)`; FKs `@ManyToOne` obrigatórias para `Evento` e `Usuario`.

Reembolso
Reembolso.java
	`id : Integer`
	`ingresso : Ingresso`
	`motivo : String`
	`status : StatusReembolso`
	`valor : BigDecimal`
	`dataSolicitacao / dataDecisao`
		PK (`IDENTITY`); vínculo exclusivo `@OneToOne` com `Ingresso` (`@JoinColumn(unique = true)`); `motivo` obrigatório (5 a 500 chars); `status` via `@Enumerated(STRING)`; registro temporal de abertura e decisão.

> **Regra de Domínio para Controle de Vagas (StatusIngresso.java)**
> A enumeração `StatusIngresso` declara a lista constante `OCUPAM_VAGA = List.of(CONFIRMADO, REEMBOLSO_SOLICITADO, UTILIZADO)`, garantindo que apenas ingressos com status `REEMBOLSADO` liberem automaticamente a vaga na capacidade do evento.

4. Matriz de Relacionamentos e Multiplicidades

Associação (Origem → Destino)	Multiplicidade	Anotação JPA	Semântica de Negócio no Conecta Event
`Evento` → `Usuario`	`0..*` para `1`	`@ManyToOne` (`organizador`)	Um usuário com perfil `ORGANIZADOR` (Perfil.java) pode publicar múltiplos eventos.
`Evento` → `Categoria`	`0..*` para `1`	`@ManyToOne` (`categoria`)	Cada evento é classificado obrigatoriamente em uma única categoria temática.
`Ingresso` → `Evento`	`0..*` para `1`	`@ManyToOne` (`evento`)	Um evento emite múltiplos ingressos até o limite definido no atributo `capacidade`.
`Ingresso` → `Usuario`	`0..*` para `1`	`@ManyToOne` (`participante`)	Um usuário participante pode adquirir ingressos para diferentes eventos cadastrados.
`Reembolso` → `Ingresso`	`0..1` para `1`	`@OneToOne` (`unique = true`)	Cada ingresso comporta no máximo uma solicitação de reembolso (StatusReembolso.java).

5. Correspondência com Modelos e Serviços do Frontend Angular

Entidade / Modelo TS	Serviço Angular	Operações e Endpoints Principais
`Usuario` / `Perfil` auth.service.ts	`AuthService` e `perfilGuard` (perfil.guard.ts)	Cadastro de conta (`POST /usuarios/cadastro`), login (`GET /usuarios/me`), autenticação via cabeçalho HTTP Basic (auth.interceptor.ts) e controle de acesso às telas por perfil.
`Categoria` categoria.model.ts	`CategoriaService` (categoria.service.ts)	`listar()` (`GET /categorias`), `salvar()` (`POST/PUT /categorias`) e `excluir()` (`DELETE /categorias/{id}`).
`Evento` / `VagasEvento` evento.model.ts	`EventoService` (evento.service.ts)	`listar()`, `listarFuturos()`, `buscarPorId()`, `vagas()` (`GET /eventos/{id}/vagas`), `salvar()`, `atualizar()` e `excluir()`.
`Ingresso` / `Reembolso` ingresso.model.ts	`IngressoService` (ingresso.service.ts)	`comprar()` (`POST /ingressos`), `listarDoUsuario()`, `atualizarTitular()` (`PUT /ingressos/{id}`), `solicitarReembolso()` (`POST /reembolsos`), `aprovarReembolso()` e `recusarReembolso()`.